

Table of contents

Multi-type Populations	1
Introduction	1
Setup	2
Multivariate PGFs	2
PGF properties	3
Two-type SIR model	3
Building the model	4
Solving the two-type model	6
Extracting population trajectories	9
Assortative mixing with a contact matrix	13
Scaling to more types	18
Summary	21
Simulation validation	21

Multi-type Populations

Simon Frost 2026-05-13

- [Introduction](#)
- [Setup](#)
- [Multivariate PGFs](#)
 - [PGF properties](#)
- [Two-type SIR model](#)
 - [Building the model](#)
- [Solving the two-type model](#)
 - [Extracting population trajectories](#)
- [Assortative mixing with a contact matrix](#)
- [Scaling to more types](#)
- [Summary](#)
- [Simulation validation](#)

Introduction

Real populations are rarely homogeneous. Age groups, risk groups, spatial patches, and other forms of heterogeneity lead to structured contact patterns where individuals of different types mix

at different rates. Multi-type edge-based compartmental models extend the EBCM framework to track the epidemic on each type of edge separately.

For K population types and M disease stages, the package automatically generates $K^2(1+M)+KM$ ODEs — the K^2 comes from tracking θ_{jl} (probability that an edge from type l to type j has not transmitted) for every ordered pair of types, K^2M from the ϕ variables for each disease stage and type pair, and KM from the per-stage population trackers (one per stage and type). This automates what would be extremely tedious bookkeeping by hand.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
```

Multivariate PGFs

In a multi-type network, each type l has its own probability generating function $\psi_l(x_1, \dots, x_K)$ that describes its connections to all K types. For a type- l node, the variable x_k corresponds to edges connecting to type- k neighbors. The PGF encodes the joint degree distribution — the probability that a randomly chosen type- l node has d_1 type-1 neighbors, d_2 type-2 neighbors, etc.

For Poisson degree distributions (the simplest case), the multivariate PGF takes the form:

$$\psi_l(x_1, \dots, x_K) = \exp\left(\sum_{k=1}^K \kappa_{lk}(x_k - 1)\right)$$

where κ_{lk} is the mean number of type- k neighbors that a type- l node has.

Let's define a two-type network with “Young” and “Old” populations:

```
@parameters κ_YY κ_YO κ_OY κ_OO
pgf_Y = multivariate_poisson_pgf([:Young, :Old], Dict{:Young => κ_YY, :Old => κ_YO})
pgf_O = multivariate_poisson_pgf([:Young, :Old], Dict{:Young => κ_OY, :Old => κ_OO})
```

```
MultivariatePGF([:Young, :Old], Any[z_Young, z_Old], exp((-1 + z_Old)*κ_OO + (-1 + z_Young)*κ_OY))
```

Here κ_{YY} is the mean number of Young neighbors a Young individual has, κ_{Y0} is the mean number of Old neighbors a Young individual has, and so on.

PGF properties

We can compute mean degrees and partial derivatives symbolically:

```
println("Mean Young neighbors of a Young node: ", mean_degree(pgf_Y, :Young))
println("Mean Old neighbors of a Young node:   ", mean_degree(pgf_Y, :Old))
println("Mean Young neighbors of an Old node:  ", mean_degree(pgf_0, :Young))
println("Mean Old neighbors of an Old node:    ", mean_degree(pgf_0, :Old))
```

```
Mean Young neighbors of a Young node:  $\kappa_{YY}$ 
Mean Old neighbors of a Young node:    $\kappa_{Y0}$ 
Mean Young neighbors of an Old node:   $\kappa_{0Y}$ 
Mean Old neighbors of an Old node:     $\kappa_{00}$ 
```

Partial derivatives describe higher-order degree structure:

```
println("∂ψ_Y/∂x_Young = ", partial_derivative(pgf_Y, :Young, 1))
println("∂²ψ_Y/∂x_Young² = ", partial_derivative(pgf_Y, :Young, 2))
println("∂²ψ_Y/(∂x_Young ∂x_Old) = ", mixed_partial(pgf_Y, :Young, :Old))
```

```
∂ψ_Y/∂x_Young = exp((-1 + z_Old)*κ_Y0 + (-1 + z_Young)*κ_YY)*κ_YY
∂²ψ_Y/∂x_Young² = exp((-1 + z_Old)*κ_Y0 + (-1 + z_Young)*κ_YY)*(κ_YY^2)
∂²ψ_Y/(∂x_Young ∂x_Old) = exp((-1 + z_Old)*κ_Y0 + (-1 + z_Young)*κ_YY)*κ_Y0*κ_YY
```

We can also evaluate the PGF at specific points:

```
println("ψ_Y(1, 1) = ", eval_multivariate_pgf(pgf_Y, Dict{:Young => 1, :Old => 1}))
```

```
ψ_Y(1, 1) = 1
```

Two-type SIR model

Now we combine the multivariate PGFs with a disease progression to build a multi-type edge-based model. We'll use a standard SIR progression:

```

@parameters  $\beta$   $\gamma$ 
progression = DiseaseProgression(
    [
        DiseaseStage(:I; transmission_rate =  $\beta$ ),
        DiseaseStage(:R; transmission_rate = 0),
    ],
    [DiseaseTransition(:I, :R,  $\gamma$ )];
    entry = :I,
)

```

```

DiseaseProgression(:S, :I, DiseaseStage[DiseaseStage(:I,  $\beta$ ), DiseaseStage(:R, 0)], DiseaseTr

```

Building the model

The `MultiTypeConfigurationModel` constructor takes the list of types, one PGF per type, and the disease progression:

```

model = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict(:Young  $\Rightarrow$  pgf_Y, :Old  $\Rightarrow$  pgf_0),
    progression = progression,
)
result = build_edge_system(model)

```

```

EdgeModelSystem(Model multitype_ebm:
Equations (16):
    16 standard: see equations(multitype_ebm)
Unknowns (16): see unknowns(multitype_ebm)
    pop_R_Old(t)
    pop_I_Old(t)
    pop_R_Young(t)
    pop_I_Young(t)
    ?
Parameters (8): see parameters(multitype_ebm)
     $\kappa_{YY}$ 
     $\kappa_{Y0}$ 
     $\rho_{Young}$ 
     $\kappa_{00}$ 
    ?

```

Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(: $\varphi_{I_Old_Old}$ => $\varphi_{I_Old_Old}(t)$,
 $1 + \theta_{Old_Young}(t) * \kappa_{Y0} + (-1 + \theta_{Young_Young}(t) * \kappa_{YY})$), (entry = pop_I_Old(t), susceptible
 $1 + \theta_{Old_Old}(t) * \kappa_{O0} + (-1 + \theta_{Young_Old}(t) * \kappa_{OY})$), :edge_seed_groups => Any[(entry = $\varphi_{I_}$
 $1 + \theta_{Old_Young}(t) * \kappa_{Y0} + (-1 + \theta_{Young_Young}(t) * \kappa_{YY}) * (1 - \rho_{Young})$), (entry = $\varphi_{I_Young_O}$
 $1 + \theta_{Old_Young}(t) * \kappa_{Y0} + (-1 + \theta_{Young_Young}(t) * \kappa_{YY}) * (1 - \rho_{Young})$), (entry = $\varphi_{I_Old_You}$
 $1 + \theta_{Old_Old}(t) * \kappa_{O0} + (-1 + \theta_{Young_Old}(t) * \kappa_{OY}) * (1 - \rho_{Old})$), (entry = $\varphi_{I_Old_Old}(t)$, ph
 $1 + \theta_{Old_Old}(t) * \kappa_{O0} + (-1 + \theta_{Young_Old}(t) * \kappa_{OY}) * (1 - \rho_{Old})$))])

Let's inspect what the model builder generated:

```
n_eqs = length(ModelingToolkit.equations(result.system))
println("Number of equations: ", n_eqs)
```

Number of equations: 16

With $K = 2$ types and $M = 2$ disease stages (I, R), we expect $K^2(1 + M) + KM = 4 \times 3 + 2 \times 2 = 16$ equations.

```
println("State variables:")
for (name, var) in sort(collect(result.variables), by = x → string(x[1]))
    println("  ", name, " → ", var)
end
```

State variables:

```
R_Old → pop_R_Old(t)
R_Young → pop_R_Young(t)
pop_I_Old → pop_I_Old(t)
pop_I_Young → pop_I_Young(t)
pop_R_Old → pop_R_Old(t)
pop_R_Young → pop_R_Young(t)
θ_Old_Old → θ_Old_Old(t)
θ_Old_Young → θ_Old_Young(t)
θ_Young_Old → θ_Young_Old(t)
θ_Young_Young → θ_Young_Young(t)
 $\varphi_{I\_Old\_Old}$  →  $\varphi_{I\_Old\_Old}(t)$ 
 $\varphi_{I\_Old\_Young}$  →  $\varphi_{I\_Old\_Young}(t)$ 
 $\varphi_{I\_Young\_Old}$  →  $\varphi_{I\_Young\_Old}(t)$ 
 $\varphi_{I\_Young\_Young}$  →  $\varphi_{I\_Young\_Young}(t)$ 
 $\varphi_{R\_Old\_Old}$  →  $\varphi_{R\_Old\_Old}(t)$ 
 $\varphi_{R\_Old\_Young}$  →  $\varphi_{R\_Old\_Young}(t)$ 
 $\varphi_{R\_Young\_Old}$  →  $\varphi_{R\_Young\_Old}(t)$ 
 $\varphi_{R\_Young\_Young}$  →  $\varphi_{R\_Young\_Young}(t)$ 
```

```
println("\nObservable quantities:")
for (name, obs) in sort(collect(result.observables), by = x → string(x[1]))
    println("  ", name)
end
```

Observable quantities:

```
I_Old
I_Young
S_Old
S_Young
edge_hazard_Old_Old
edge_hazard_Old_Young
edge_hazard_Young_Old
edge_hazard_Young_Young
excess_hazard_Old_Old
excess_hazard_Old_Young
excess_hazard_Young_Old
excess_hazard_Young_Young
φ_S_Old_Old
φ_S_Old_Young
φ_S_Young_Old
φ_S_Young_Young
```

The variables follow a naming convention:

- θ_{jl} ($\theta_{\text{Young_Young}}$, $\theta_{\text{Old_Young}}$, etc.): probability that an edge from type l to type j has not transmitted infection
- $\phi_{m,jl}$ ($\phi_{\text{I_Young_Young}}$, $\phi_{\text{R_Old_Young}}$, etc.): probability that a type- j neighbor reached via a type- l edge is in disease stage m
- R_l (R_{Young} , R_{Old}): fraction of type- l population that has recovered

Solving the two-type model

We assign numeric parameter values representing a contact structure where Young individuals have more contacts (especially with each other), and Old individuals have fewer contacts overall:

Parameter	Value	Meaning
κ_{YY}	6	Young $\rightarrow$ Young contacts
κ_{YO}	2	Young $\rightarrow$ Old contacts
κ_{OY}	2	Old $\rightarrow$ Young contacts

Parameter	Value	Meaning
κ_{OO}	4	Old $\Rightarrow$ Old contacts
γ	0.25	Recovery rate
T	0.75	Edge transmissibility
β	0.75	Transmission rate ($\beta = T\gamma/(1 - T)$)

Canonical anchor. This vignette uses $\gamma = 0.25$ and per-edge transmissibility $T = \beta/(\beta + \gamma) = 0.75$, giving $\beta = T\gamma/(1 - T) = 0.75$. $R_0 = T\rho(M)$ where M is the next-generation matrix derived from the contact structure; preserving T preserves R_0 .

```

κ_YY_val = 6.0
κ_YO_val = 2.0
κ_OY_val = 2.0
κ_OO_val = 4.0
γ_val = 0.25
T_val = 0.75
β_val = T_val * γ_val / (1 - T_val)
ε = 1e-2
tspan = (0.0, 40.0)

```

(0.0, 40.0)

Set up initial conditions and parameters:

```

ic = default_initial_conditions(result; ε = ε)
params = Dict(
    κ_YY ⇒ κ_YY_val, κ_YO ⇒ κ_YO_val,
    κ_OY ⇒ κ_OY_val, κ_OO ⇒ κ_OO_val,
    β ⇒ β_val, γ ⇒ γ_val,
)

prob = ODEProblem(result.system, merge(ic, params), tspan)
sol = solve(prob, Tsit5())

```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 36-element Vector{Float64}:

0.0

0.06003976573389654


```
[0.9881194508088355, 0.00044764277499854336, 0.9970357454332888, 0.0004294961899349146, 0.24
10, 3.10123863144964e-10, 3.3699512210557434e-10, 3.3699512210557434e-
10, 0.25857468004471745, 0.25857468004471745, 0.25190106903532894, 0.25190106903532894]
[0.9883865606029001, 0.0001805329814652635, 0.9972920270965151, 0.0001732145263112047, 0.24
10, 2.7107097089715446e-10, 3.847617880565842e-10, 3.847617880565842e-
10, 0.25857468001502926, 0.25857468001502926, 0.25190106907145193, 0.25190106907145193]
[0.9885005886193382, 6.650495217327531e-5, 0.9974014326281276, 6.380898913797434e-
5, 0.24714177328922388, 0.24714177328922388, 0.2493663102002047, 0.2493663102002047, 4.14615
10, 4.146157442883999e-10, 8.16446548029662e-10, 8.16446548029662e-
10, 0.25857468013232826, 0.25857468013232826, 0.251901069399386, 0.251901069399386]
[0.9885059015561801, 6.119202337161398e-5, 0.9974065301901142, 5.871143023431761e-
5, 0.24714177331890017, 0.24714177331890017, 0.24936631025814193, 0.24936631025814193, 3.039
10, 3.0395083113483007e-10, 5.877804586668547e-10, 5.877804586668547e-
10, 0.2585746800432994, 0.2585746800432994, 0.25190106922557426, 0.25190106922557426]
```

Extracting population trajectories

For Poisson PGFs, we can compute the susceptible fraction of each type from the θ values. For type l :

$$S_l(t) = \psi(\theta_{1l}(t), \dots, \theta_{Kl}(t)) = \exp\left(\sum_{k=1}^K \kappa_{lk}(\theta_{kl}(t) - 1)\right)$$

```
# Extract  $\theta$  trajectories
 $\theta_{YY}$  = compartment(sol, result, : $\theta_{\text{Young\_Young}}$ )
 $\theta_{OY}$  = compartment(sol, result, : $\theta_{\text{Old\_Young}}$ )
 $\theta_{YO}$  = compartment(sol, result, : $\theta_{\text{Young\_Old}}$ )
 $\theta_{OO}$  = compartment(sol, result, : $\theta_{\text{Old\_Old}}$ )

# Compute S for each type using the Poisson PGF formula
 $S_{\text{Young}}$  = exp.( $\kappa_{YY\_val} \cdot (\theta_{YY} - 1) + \kappa_{YO\_val} \cdot (\theta_{OY} - 1)$ )
 $S_{\text{Old}}$  = exp.( $\kappa_{OY\_val} \cdot (\theta_{YO} - 1) + \kappa_{OO\_val} \cdot (\theta_{OO} - 1)$ )

# Extract R for each type
 $R_{\text{Young}}$  = compartment(sol, result, : $R_{\text{Young}}$ )
 $R_{\text{Old}}$  = compartment(sol, result, : $R_{\text{Old}}$ )

#  $I = 1 - S - R$ 
 $I_{\text{Young}}$  = 1.0 -  $S_{\text{Young}}$  -  $R_{\text{Young}}$ 
 $I_{\text{Old}}$  = 1.0 -  $S_{\text{Old}}$  -  $R_{\text{Old}}$ 
```

36-element Vector{Float64}:

```
0.0
0.0028755601275356176
0.0081377451919941
0.017269693266252235
0.03441630122532625
0.06741503478371039
0.13316197368578997
0.2629696294017459
0.4557110283117928
0.6269002820387685
0.816423494838608832
0.909014527806844641
0.9455920311298097
0.962154195691964511
0.979274554952097835
0.993401236325273338
7.301383899949165e-5
-4.101419042235044e-5
-4.6327119142874906e-5
```

```
plot(sol.t, S_Young, label="S Young", lw=2, color=1)
plot!(sol.t, I_Young, label="I Young", lw=2, color=2)
plot!(sol.t, R_Young, label="R Young", lw=2, color=3)
plot!(sol.t, S_Old, label="S Old", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, ls=:dash, color=2)
plot!(sol.t, R_Old, label="R Old", lw=2, ls=:dash, color=3)
xlabel!("Time")
ylabel!("Fraction")
title!("Two-type SIR: Young (solid) vs Old (dashed)")
```

The Young population (solid lines) experiences a faster epidemic wave because they have more total contacts ($\kappa_{YY} + \kappa_{YO} = 8$) compared to the Old population ($\kappa_{OY} + \kappa_{OO} = 6$, dashed lines).

```
plot(sol.t, I_Young, label="I Young", lw=2, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Infection prevalence by type")
```

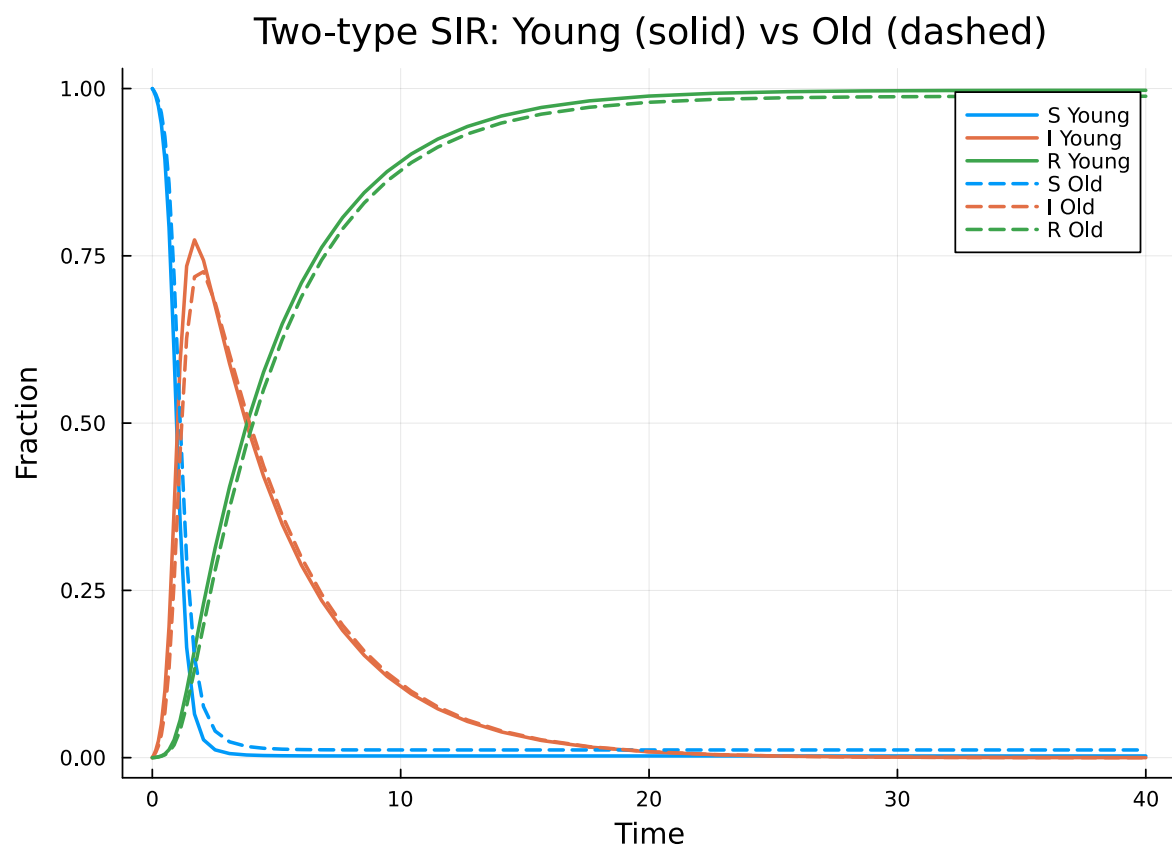


Figure 1: Figure 1: SIR dynamics in a two-type (Young/Old) population. Young individuals have more contacts and experience a faster, larger epidemic.

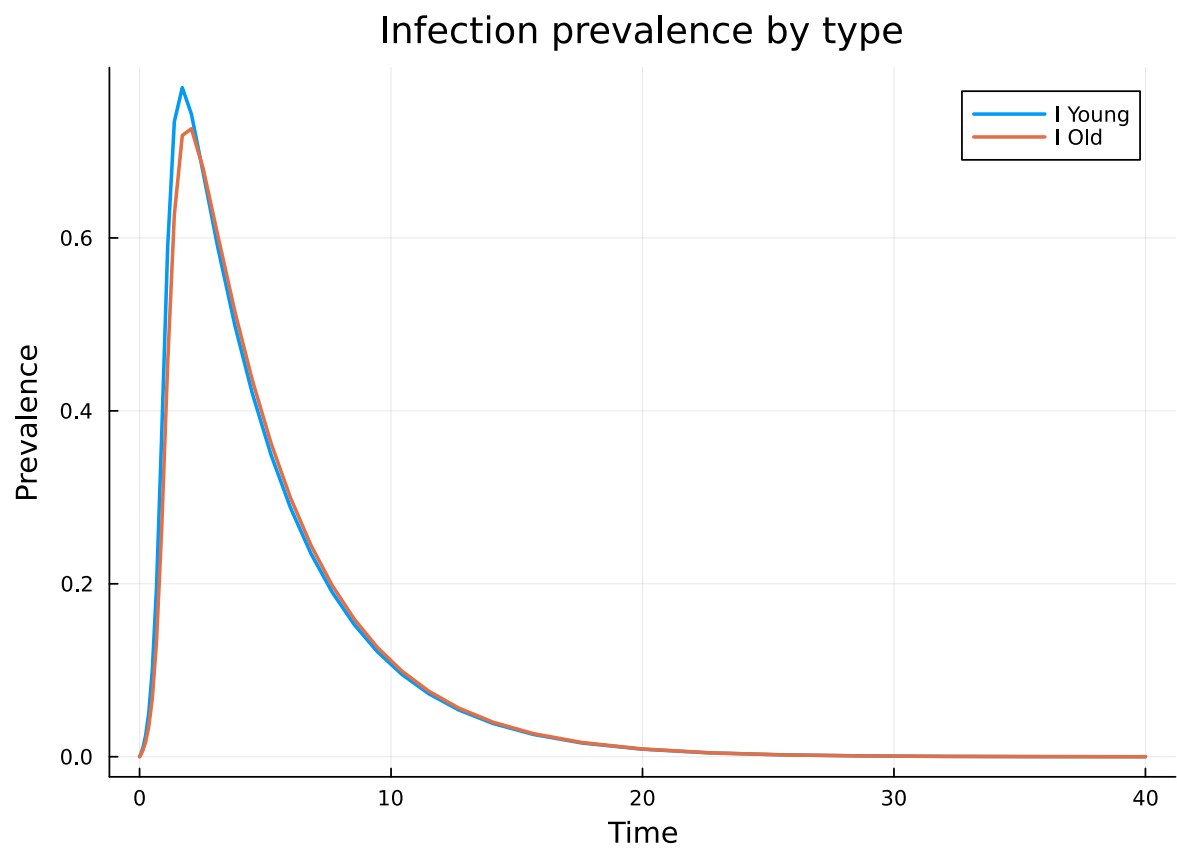


Figure 2: Figure 2: Prevalence (infected fraction) comparison between Young and Old populations.

Assortative mixing with a contact matrix

By default, the multi-type model assumes homogeneous mixing — the transmission rate across an edge depends only on the disease state of the infector. In reality, mixing is often assortative: individuals preferentially contact others of the same type.

The `contact_matrix` parameter scales the transmission rate on each type of edge. A value of 1.0 means the baseline transmission rate applies; values less than 1.0 reduce cross-group transmission:

```
model_assort = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict(:Young => pgf_Y, :Old => pgf_O),
    progression = progression,
    contact_matrix = Dict(
        (:Young, :Young) => 1.0,
        (:Old, :Old) => 1.0,
        (:Young, :Old) => 0.5,
        (:Old, :Young) => 0.5,
    ),
)
result_assort = build_edge_system(model_assort)
```

EdgeModelSystem(Model multitype_ebm:

Equations (16):

16 standard: see equations(multitype_ebm)

Unknowns (16): see unknowns(multitype_ebm)

pop_R_Old(t)

pop_I_Old(t)

pop_R_Young(t)

pop_I_Young(t)

?

Parameters (8): see parameters(multitype_ebm)

κ_{YY}

κ_{YO}

ρ_{Young}

κ_{OO}

?

Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(: $\varphi_{I_Old_Old}$ => $\varphi_{I_Old_Old}(t)$,
1 + $\theta_{Old_Young}(t)$)* κ_{YO} + (-1 + $\theta_{Young_Young}(t)$)* κ_{YY}), (entry = pop_I_Old(t), susceptible
1 + $\theta_{Old_Old}(t)$)* κ_{OO} + (-1 + $\theta_{Young_Old}(t)$)* κ_{OY})), :edge_seed_groups => Any[(entry = $\varphi_{I_}$
1 + $\theta_{Old_Young}(t)$)* κ_{YO} + (-1 + $\theta_{Young_Young}(t)$)* κ_{YY})*(1 - ρ_{Young})], (entry = $\varphi_{I_Young_Old}$

```
1 +  $\theta_{\text{Old\_Young}}(t)$ ))* $\kappa_{\text{Y0}}$  + (-1 +  $\theta_{\text{Young\_Young}}(t)$ ))* $\kappa_{\text{YY}}$ )*(1 -  $\rho_{\text{Young}}$ )), (entry =  $\varphi_{\text{I\_Old\_Young}}$ 
1 +  $\theta_{\text{Old\_Old}}(t)$ ))* $\kappa_{\text{O0}}$  + (-1 +  $\theta_{\text{Young\_Old}}(t)$ ))* $\kappa_{\text{OY}}$ )*(1 -  $\rho_{\text{Old}}$ )), (entry =  $\varphi_{\text{I\_Old\_Old}}(t)$ , ph
1 +  $\theta_{\text{Old\_Old}}(t)$ ))* $\kappa_{\text{O0}}$  + (-1 +  $\theta_{\text{Young\_Old}}(t)$ ))* $\kappa_{\text{OY}}$ )*(1 -  $\rho_{\text{Old}}$ ))]]))
```

The contact matrix entry $c_{jl} = 0.5$ for cross-type edges means that a Young infector transmits to an Old neighbor at half the baseline rate (and vice versa). This models reduced cross-group transmission efficiency due to shorter or less intimate contacts.

```
ic_assort = default_initial_conditions(result_assort;  $\varepsilon = \varepsilon$ )
prob_assort = ODEProblem(result_assort.system, merge(ic_assort, params), tspan)
sol_assort = solve(prob_assort, Tsit5())
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 35-element Vector{Float64}:

0.0

0.06309194293588453

0.15518114446885634

0.2650483578237706

0.40084076888262965

0.5601325273621369

0.7489955757295323

0.9795372155732923

1.2560479793654462

1.5541195967806871

⋮

17.293558431035045

19.13729898133455

21.26228044093317

23.748545480062603

26.582640437384583

29.64666803245552

32.97412240481453

36.59835456681955

40.0

u: 35-element Vector{Vector{Float64}}:

[0.0, 0.01, 0.0, 0.01, 0.0, 0.0, 0.0, 0.0, 0.010000000000000009, 0.010000000000000009, 0.0100

[0.00017611162505556173, 0.012407883336372846, 0.000184510845865112, 0.013541225922849556,

[0.0005107899557458583, 0.016879317428765087, 0.0005720843695279228, 0.0205390354801317, 0.

[0.0010692971439301485, 0.024211200050781086, 0.0012929847631462787, 0.03278414963090177, 0.

[0.0021024060233171137, 0.03760710801929742, 0.0027759277437838853, 0.05646155281665524, 0.

[0.004056461702881903, 0.06254535808368321, 0.0058572115811865175, 0.10212640008988638, 0.0

[0.00807087130360951, 0.11172454282785116, 0.012619485666443558, 0.19142256374115213, 0.000
 [0.01714729408959914, 0.21086702966848545, 0.02812481056842392, 0.35495623292060885, 0.0142
 [0.037307662316766015, 0.37762724643953455, 0.06026525736873552, 0.5689113617611292, 0.0303
 [0.07215815800862171, 0.5495222777183052, 0.10868321144904866, 0.7106012854471992, 0.056417
 7
 [0.9651606818955317, 0.019208179931967977, 0.9787315479670317, 0.017827537430031487, 0.2460
 6, 6.474190739406303e-5, 4.677537605221957e-5, 5.969280734310598e-7, 0.2617248240138474, 0.4
 [0.9722545255135835, 0.012115034768497273, 0.9853153947834551, 0.01124388356903432, 0.24609
 7, 2.0856351569389833e-5, 1.4904605792080235e-5, 1.7075753591360037e-
 7, 0.26172326667519635, 0.40939077764169324, 0.40207337575198154, 0.25258066930378503]
 [0.977247490975839, 0.00712230631359978, 0.9899492915192234, 0.006610053276603067, 0.246092
 7, 5.667660019809957e-6, 4.002412257975838e-6, 4.3417709796141674e-
 8, 0.2617227646344853, 0.4093815222223487, 0.4020667945698589, 0.2525805239664127]
 [0.9805443452152104, 0.0038255198543174548, 0.993009014399835, 0.0035503496765892997, 0.246
 8, 1.2481790494738053e-6, 8.705478993594412e-7, 9.388243764086949e-
 9, 0.2617226253391991, 0.409378829865713, 0.402064903882885, 0.2525804839838648]
 [0.9824862140385825, 0.0018836664197423455, 0.9948112007259439, 0.0017481677900852415, 0.24
 9, 2.2942748940779155e-7, 1.5824093194522139e-7, 1.7891933595109228e-
 9, 0.26172259410608867, 0.40937820938149877, 0.4020644738349417, 0.2525804749548734]
 [0.9834941606175704, 0.0008757226945516769, 0.9957466414974985, 0.000812727851529338, 0.246
 9, 3.8316621409176464e-8, 2.6206253902881563e-8, 3.446704456598285e-
 10, 0.2617225882988753, 0.4093780930026996, 0.4020643941143357, 0.2525804732467322]
 [0.9839886803253605, 0.0003812034660832048, 0.9962055879449154, 0.00035378154523539173, 0.2
 10, 5.962888742907263e-9, 4.067947060166621e-9, 9.258850515900184e-
 11, 0.26172258731242337, 0.4093780733028671, 0.40206438074667783, 0.25258047295182856]
 [0.9842157909851124, 0.00015409288056566413, 0.9964163613444526, 0.00014300816754160794, 0.
 11, 9.178980210826275e-10, 6.328432534052354e-10, 6.35061001982231e-
 11, 0.261722587161582, 0.40937807023133205, 0.40206437867250955, 0.2525804729136343]
 [0.9843040379567242, 6.584592161778135e-5, 0.9964982602418259, 6.110927378762873e-
 5, 0.2460924709596559, 0.39374795349620567, 0.3986237477677978, 0.2491398423679308, 3.971790
 11, 1.3782765486217007e-10, 9.611862474385797e-11, 4.38903192399519e-
 11, 0.26172258712103214, 0.40937806975569146, 0.4020643783483032, 0.252580472896208]

```
# Extract trajectories for the assortative model
θ_YY_a = compartment(sol_assort, result_assort, :θ_Young_Young)
θ_OY_a = compartment(sol_assort, result_assort, :θ_Old_Young)
θ_YO_a = compartment(sol_assort, result_assort, :θ_Young_Old)
θ_OO_a = compartment(sol_assort, result_assort, :θ_Old_Old)

S_Young_a = exp.(κ_YY_val .* (θ_YY_a .- 1) .+ κ_YO_val .* (θ_OY_a .- 1))
S_Old_a = exp.(κ_OY_val .* (θ_YO_a .- 1) .+ κ_OO_val .* (θ_OO_a .- 1))
R_Young_a = compartment(sol_assort, result_assort, :R_Young)
```

```
R_Old_a = compartment(sol_assort, result_assort, :R_Old)
I_Young_a = 1.0 .- S_Young_a .- R_Young_a
I_Old_a = 1.0 .- S_Old_a .- R_Old_a
```

35-element Vector{Float64}:

```
0.0
0.0024339842900055553
0.006953964849552543
0.01436554760616001
0.027907196600209253
0.05311704446697316
0.10283329119624385
0.2030680494411372
0.3717187849980235
0.5456957597410426
␣
0.019057795456768556
0.01196465734894303
0.006971931288185407
0.0036751455135713984
0.001733292234440298
0.0007253485380759317
0.00023082931444906585
3.7187296814122917e-6
-8.452822913862512e-5
```

```
plot(sol.t, I_Young, label="I Young (homogeneous)", lw=2, color=1)
plot!(sol_assort.t, I_Young_a, label="I Young (assortative)", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old (homogeneous)", lw=2, color=2)
plot!(sol_assort.t, I_Old_a, label="I Old (assortative)", lw=2, ls=:dash, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Homogeneous (solid) vs assortative (dashed) mixing")
```

Assortative mixing tends to slow the epidemic because cross-group transmission is a major driver of epidemic spread — reducing it isolates the two populations, making the overall dynamics more like two separate, smaller epidemics.

```
final_R_Young_hom = R_Young[end]
final_R_Old_hom = R_Old[end]
final_R_Young_assort = R_Young_a[end]
```

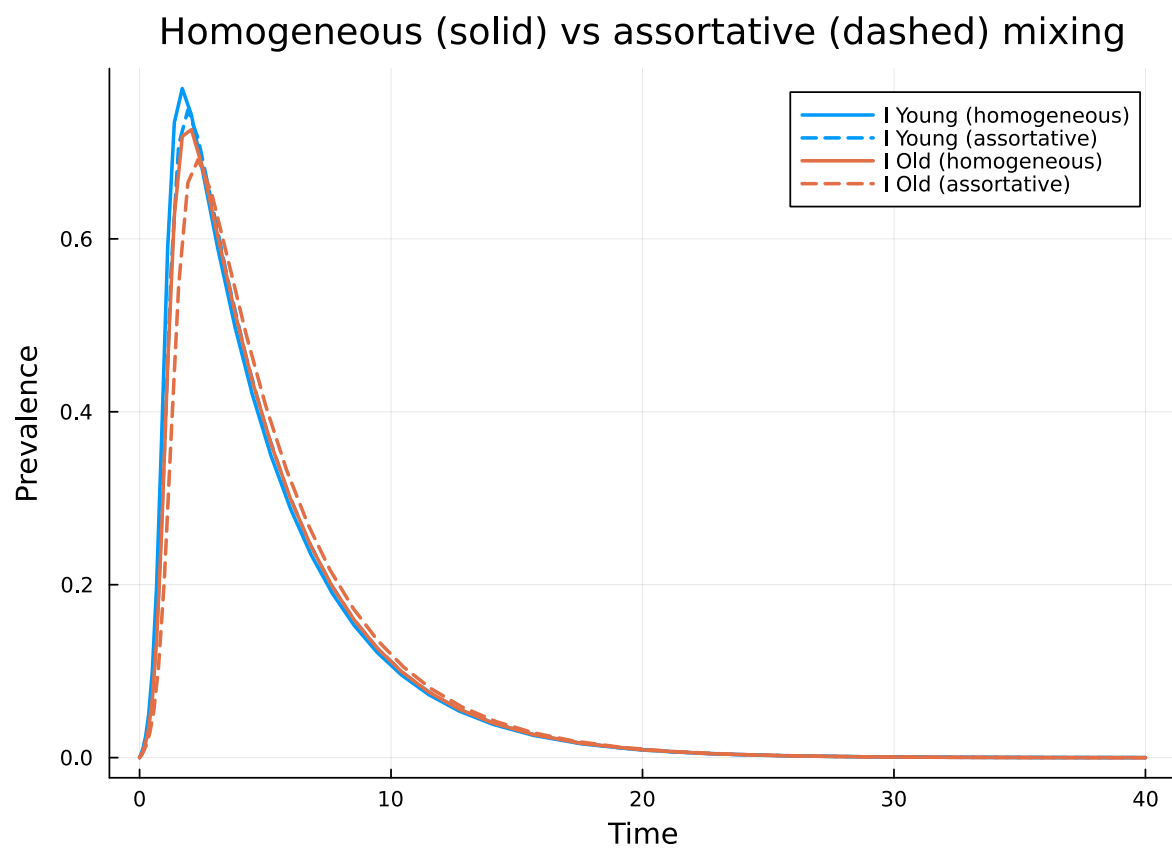



Figure 3: Figure 3: Effect of assortative mixing on epidemic dynamics. Assortative mixing (dashed) slows the epidemic and reduces final size compared to homogeneous mixing (solid).

```

final_R_Old_assort = R_Old_a[end]

bar_labels = ["Young\n(homogeneous)", "Young\n(assortative)", "Old\n(homogeneous)", "Old\n(assortative)"]
bar_vals = [final_R_Young_hom, final_R_Young_assort, final_R_Old_hom, final_R_Old_assort]
bar(bar_labels, bar_vals, legend=False, ylabel="Final size ( $R_\infty$ )",
    title="Final epidemic size by type and mixing", color=[1 1 2 2],
    ylim=(0, 1))

```

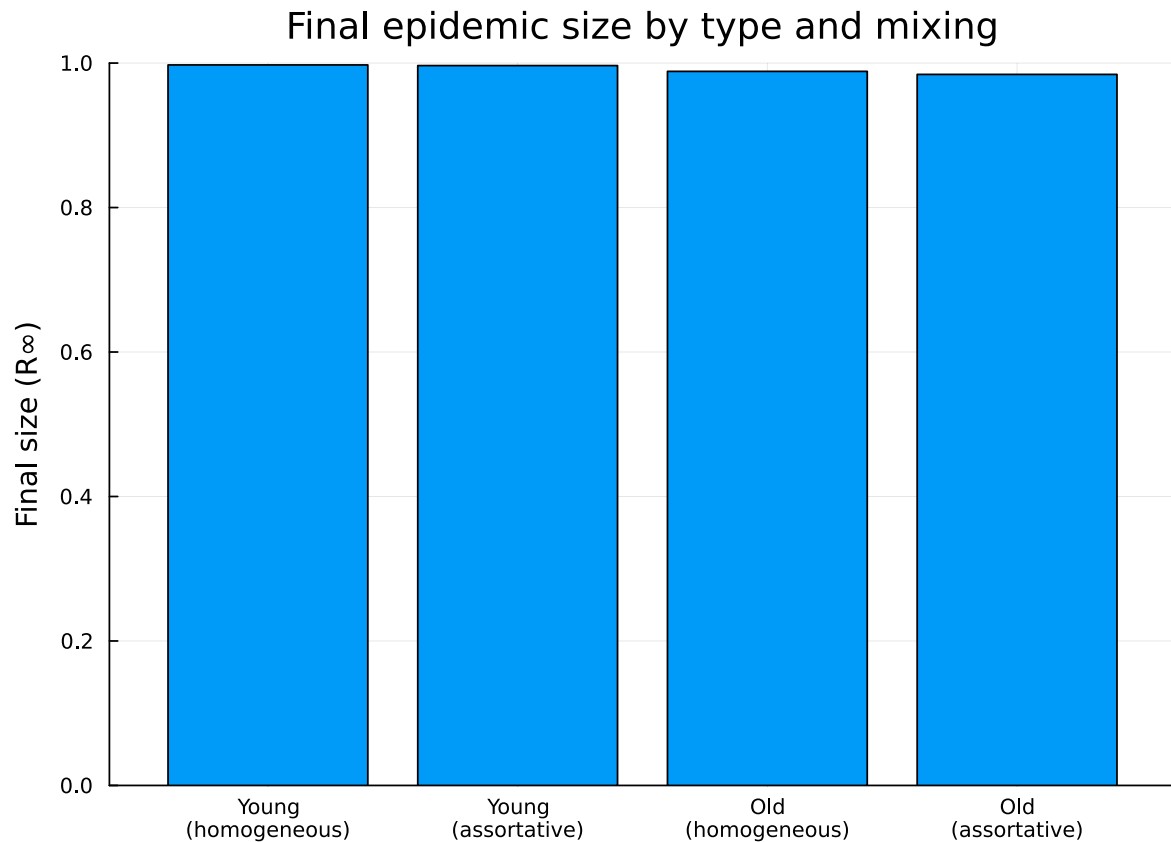


Figure 4: Figure 4: Final epidemic size (recovered fraction) under homogeneous vs assortative mixing.

Scaling to more types

The multi-type framework scales to any number of population types. For example, consider a three-type (Young/Middle/Old) SEIR model:

```
@parameters  $\sigma$ 
```

```
progression_seir = DiseaseProgression(  
  [  
    DiseaseStage(:E; transmission_rate = 0),  
    DiseaseStage(:I; transmission_rate =  $\beta$ ),  
    DiseaseStage(:R; transmission_rate = 0),  
  ],  
  [  
    DiseaseTransition(:E, :I,  $\sigma$ ),  
    DiseaseTransition(:I, :R,  $\gamma$ ),  
  ];  
  entry = :E,  
)
```

DiseaseProgression(:S, :E, DiseaseStage[DiseaseStage(:E, 0), DiseaseStage(:I, β), DiseaseSta

```
@parameters  $\kappa_{YY3}$   $\kappa_{YM}$   $\kappa_{Y03}$   $\kappa_{MY}$   $\kappa_{MM}$   $\kappa_{M0}$   $\kappa_{OY3}$   $\kappa_{OM}$   $\kappa_{003}$ 
```

```
types_3 = [:Y, :M, :O]
```

```
pgf_Y3 = multivariate_poisson_pgf(types_3, Dict(:Y  $\Rightarrow$   $\kappa_{YY3}$ , :M  $\Rightarrow$   $\kappa_{YM}$ , :O  $\Rightarrow$   $\kappa_{Y03}$ ))
```

```
pgf_M3 = multivariate_poisson_pgf(types_3, Dict(:Y  $\Rightarrow$   $\kappa_{MY}$ , :M  $\Rightarrow$   $\kappa_{MM}$ , :O  $\Rightarrow$   $\kappa_{M0}$ ))
```

```
pgf_O3 = multivariate_poisson_pgf(types_3, Dict(:Y  $\Rightarrow$   $\kappa_{OY3}$ , :M  $\Rightarrow$   $\kappa_{OM}$ , :O  $\Rightarrow$   $\kappa_{003}$ ))
```

```
model_3type = MultiTypeConfigurationModel(  
  types = types_3,  
  pgfs = Dict(:Y  $\Rightarrow$  pgf_Y3, :M  $\Rightarrow$  pgf_M3, :O  $\Rightarrow$  pgf_O3),  
  progression = progression_seir,  
)
```

```
result_3type = build_edge_system(model_3type)
```

```
n_eqs_3 = length(ModelingToolkit.equations(result_3type.system))  
println("Three-type SEIR: $n_eqs_3 equations automatically generated")
```

Three-type SEIR: 45 equations automatically generated

With $K = 3$ types and $M = 3$ stages (E, I, R), we expect $K^2(1 + M) + KM = 9 \times 4 + 3 \times 3 = 45$ equations. Writing these by hand would be impractical and error-prone — the package handles all the symbolic bookkeeping automatically.

```
println("\nState variables (", length(result_3type.variables), " total):")
for (name, _) in sort(collect(result_3type.variables), by = x → string(x[1]))
    println("  ", name)
end
```

State variables (48 total):

R_M
 R_O
 R_Y
 pop_E_M
 pop_E_O
 pop_E_Y
 pop_I_M
 pop_I_O
 pop_I_Y
 pop_R_M
 pop_R_O
 pop_R_Y
 θ _M_M
 θ _M_O
 θ _M_Y
 θ _O_M
 θ _O_O
 θ _O_Y
 θ _Y_M
 θ _Y_O
 θ _Y_Y
 φ _E_M_M
 φ _E_M_O
 φ _E_M_Y
 φ _E_O_M
 φ _E_O_O
 φ _E_O_Y
 φ _E_Y_M
 φ _E_Y_O
 φ _E_Y_Y
 φ _I_M_M
 φ _I_M_O
 φ _I_M_Y
 φ _I_O_M
 φ _I_O_O
 φ _I_O_Y

$\varphi_{I_Y_M}$
 $\varphi_{I_Y_O}$
 $\varphi_{I_Y_Y}$
 $\varphi_{R_M_M}$
 $\varphi_{R_M_O}$
 $\varphi_{R_M_Y}$
 $\varphi_{R_O_M}$
 $\varphi_{R_O_O}$
 $\varphi_{R_O_Y}$
 $\varphi_{R_Y_M}$
 $\varphi_{R_Y_O}$
 $\varphi_{R_Y_Y}$

Summary

Multi-type edge-based models capture the essential features of heterogeneous populations:

- Structured contact patterns: Different types can have different degree distributions and mixing preferences.
- Type-specific dynamics: Each population type experiences its own epidemic trajectory, determined by both its own contact structure and cross-group transmission.
- Contact matrices: Assortative or disassortative mixing can be specified to modulate cross-group transmission rates.
- Automatic scaling: The package generates $K^2(1+M)+KM$ equations for any number of types K and disease stages M , handling all the symbolic algebra and bookkeeping automatically.

Simulation validation

This vignette focuses on a scenario for which `NetworkOutbreaks.jl` does not yet provide an out-of-the-box stochastic ground truth (multi-type host graphs with prescribed mixing matrices, time-varying networks via the EBCM rewiring schedule, multiplex layers, degree-correlated configuration models, clustered graphs with prescribed triangle counts, or final-size sweeps over R_0).

A future revision will add the missing primitives to `NetworkOutbreaks.jl` and overlay the corresponding Gillespie SSA ribbon here. Until then, see [vignette 01](#) for the validation pattern on a single-layer Poisson configuration-model network with the same canonical parameters.